

Indian Institute of Technology

Department of Cognitive and Brain Sciences

ES 673 - NeuroAI: Neuroscience & Artificial Intelligence
Take Home Assignments

Jaskirat Singh Maskeen

(23110146 | jaskirat.maskeen@iitgn.ac.in)

Instructor: Vaibhav Tripathi

August 29, 2026

Take Home Assignment 1

Bio-Inspired Modified Perceptron

Github Repository

1.1 Introduction

As discussed in class, the Artificial Neural Networks (ANNs) we see today started with a basic building block: the perceptron. A perceptron consists of connected neurons, with weights on the connections and each neuron having its own bias. It also includes an activation function that resembles a spike in a biological neuron. However, there are many differences between an artificial neuron and a biological neuron.

For this assignment, the goal is to modify the perceptron to add biological realism. Fig. 1.1 shows how biological neural networks differ from ANNs, as discussed in class. Therefore, we will tackle some of these differences in this assignment to move closer to biological realism.

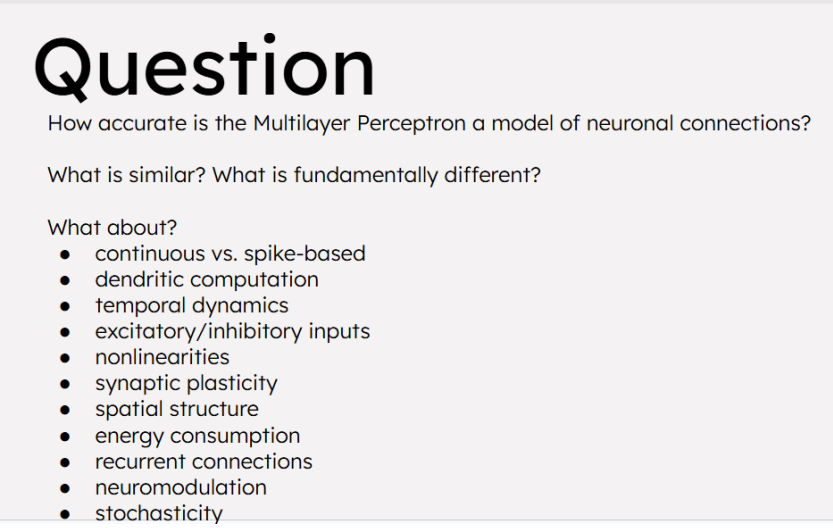


Figure 1.1: Possible avenues for Realism

One of the major differences that we do not plan to tackle in this assignment is the need for backpropagation.

> **Note:** Most of the ideas we had for including properties from 1.1 were already discovered while researching online, so we have cited those sources and kept their original names.

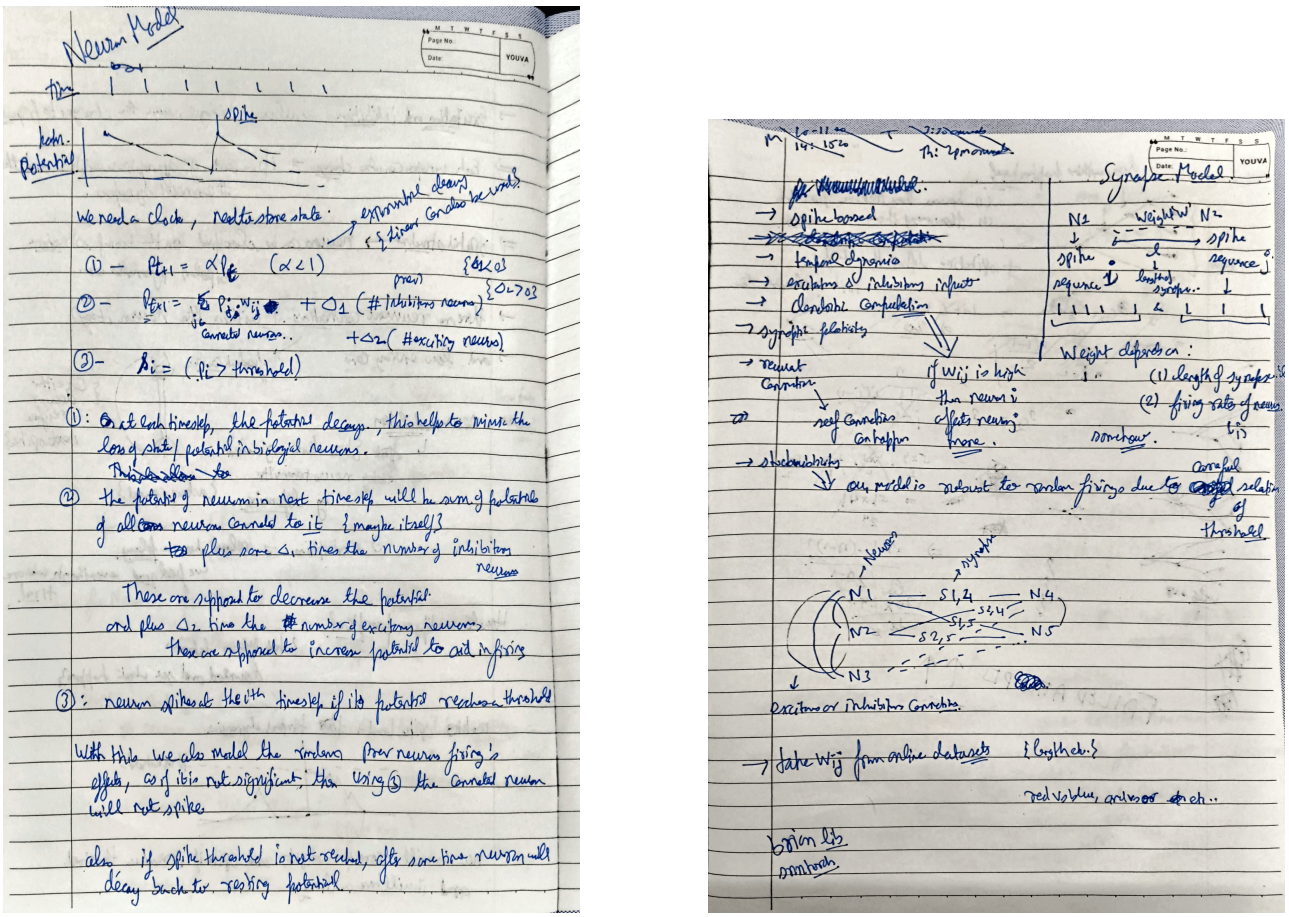
1.2 Data Acquisition

We use information from two datasets:

1. **The MICrONS Dataset:** This dataset (specifically `minnie65`) provides a millimetre-scale functional connectomics volume of the mouse visual cortex (1). From this, we extract the spatial 3D coordinates (x, y, z) of real neurons and their biological classifications (excitatory vs. inhibitory). We use this information to determine neuronal lengths (to infer spatial connections and attenuation) and to understand the distribution of excitatory and inhibitory neurons.
2. **MNIST Dataset:** This is a standard benchmarking dataset consisting of images of handwritten digits. We will use this dataset as the main problem for our perceptron to solve.

1.3 Modifications to Perceptron

In this section, we explain the modifications we made to the perceptron. Most of these ideas are taken directly from the in-class assignment (Fig. 1.2).



(a) Neuron Model

(b) Synapse Model

Figure 1.2: In Class Assignment 2

1.3.1 Leaky Integrate-and-Fire (LIF) Dynamics

We discretise the time axis and consider the neurons to have a continuous membrane potential. Incoming synaptic currents and lateral feedback directly affect the internal **membrane potential**. However, if no spikes arrive, the potential decays to the resting potential.

The following equation governs the neuron's potential:

$$P_i(t+1) = \alpha \cdot P_i(t) + (1-\alpha)[I_i^{syn}(t) + I_i^{lat}(t)] + \sigma_{noise}\sqrt{1-\alpha^2} \cdot \xi_i(t) \tag{1.1}$$

where $\alpha = \exp(-\Delta t/\tau_m)$ is the decay factor, $I_i^{syn}(t)$ is the incoming synaptic current, and $I_i^{lat}(t)$ represents lateral population feedback from the current layer. This temporal integration allows the network to maintain a short-term memory of recent stimuli.

1.3.2 Effect of length of synapse, and Synaptic Scaling

According to Cable Theory (2), signals travelling down physical dendrites decay exponentially over distance. We already have the neuron lengths from the 3D coordinates in the MICrONS dataset. Therefore, we create an attenuation mask $A_{i,j} = \exp(-d_{i,j}/\lambda)$ that suppresses distant signals using a length constant $\lambda = 150.0\mu m$, and beyond 5λ, $A_{i,j} = 0$. To avoid the vanishing gradient problem (as we are still using a variation of backpropagation for learning), we normalise this mask per neuron (resulting in A^{scaled}). This mimics homeostatic plasticity, where real cells upregulate receptors to maintain baseline stimulation. We also use a binary adjacency matrix (Adj) that serves as a mask (for PyTorch), indicating where gradients should flow.

1.3.3 Excitatory/Inhibitory Action

While reading through online sources, we found that this concept relates to Dale's Principle (3), which states that a neuron acts as either excitatory or inhibitory uniformly across all its synaptic sites. We extract the true excitatory and inhibitory neuron classifications from the Baylor cell-type dataset. A `dale_mask` (+1 for excitatory, -1 for inhibitory) is permanently multiplied by the absolute raw synaptic weights:

$$W^{eff} = |W^{raw}| \odot dale_mask \odot A^{scaled} \odot Adj$$

1.3.4 Short-Term Plasticity (STP)

This mechanism tracks the neurotransmitter pool for each synapse. During rapid spiking, this pool depletes and takes time to recharge. We use the Tsodyks-Markram model (4) to simulate this effect. We track a continuous value, $R \in [0, 1]$, representing this neurotransmitter pool:

1. Release: A spike utilizes a fraction U of this pool, $I_{release} = R \cdot U \cdot S_i(t)$. Here, U is the utilisation factor, which is a constant representing the fraction of the available neurotransmitter pool consumed by a single spike, and $S_i(t)$ is 1 if the ith neuron fires at the tth timestep.
2. Depletion: The pool is immediately depleted by the amount released.
3. Recovery: Between spikes, R exponentially recovers toward 1.0 with a time constant τ_d:

$$R(t+1) = R(t) + \frac{1 - R(t)}{\tau_d} - I_{release}$$

Ideally, this should force the network to rely on spike timings, as it prevents neurons from firing too rapidly.

1.3.5 Dynamic Thresholding

In addition to the STP mechanism, we also make the firing thresholds, $\theta_i(t)$, dynamic (5). This makes successive firings harder, preventing the rapid firing of neurons. The following equation gives the dynamics:

$$\theta_i(t+1) = \theta_0 + (\theta_i(t) - \theta_0) \exp\left(-\frac{\Delta t}{\tau_\theta}\right) + \gamma \cdot S_i(t) \tag{1.2}$$

where θ_0 is the resting threshold of the neuron. Whenever a spike occurs, the threshold's value increases by γ. The exponential term allows the threshold to decay back to the resting threshold. This acts as an implicit regulariser.

Once a spike is generated, the membrane potential drops to the resting potential immediately:

$$P_i(t+1) \leftarrow P_i(t+1) \cdot (1 - S_i(t)) + P_{rest} \cdot S_i(t) \tag{1.3}$$

1.3.6 Learning Rule

Since spikes are emitted through a unit step function, their derivatives are zero everywhere except at 0, where they tend to ∞. Therefore, we approximate spikes using a scaled sigmoid function as a surrogate, which has a defined derivative $\frac{\partial S}{\partial P} = \frac{1}{(1+k|P-\theta|)^2}$ (6; 7). We use backpropagation through time (BPTT), where the gradients are unrolled across the batch and through the timesteps for each sample.

1.4 Parameters

We use the following parameter values. Changing them could significantly affect performance:

- **The Length Constant ($\lambda = 150.0\mu m$)**: This is based on the assumed physical density of the cortical volume. The attenuation mask scales as $\exp(-d/\lambda)$. Increasing this value reduces sparsity and makes the modified perceptron behave like a fully connected one.
- **Membrane Time Constant ($\tau_m = 20$ ms)**: This controls the leakage of the neuron's accumulated potential. A very small time constant allows rapid leakage, preventing the neuron from ever spiking. A very large time constant leads to almost no leakage, causing the neuron to act as a pure integrator (similar to a linear or ReLU activation, where everything passes through).
- **Neurotransmitter Utilisation Factor ($U = 0.2$)**: This controls the fraction of the neurotransmitter pool used when a spike occurs. A high utilisation factor increases synaptic fatigue, restricting the network's ability to sustain closely spaced spikes, which can affect learning. A very low value disables the short-term plasticity effect.
- **Threshold Increase Amount ($\gamma = 0.02$)**: This controls the jump in the firing threshold once a spike occurs. If γ is too high, a single spike can make the neuron refractory, meaning it may not spike again in future timesteps until the threshold decays. If $\gamma \to 0$, the dynamic threshold effect ceases to exist, leaving only a static threshold.

1.5 Results

1.5.1 Bio-Inspired Perceptron Model

We have a three-layer Network (one input, one hidden, and one output layer) to process information temporally over discrete time steps.
Architecture:

- **Input layer**: 784 neurons, corresponding to the flattened 28×28 MNIST image.
- **Hidden layer**: 128 biologically constrained neurons.
- **Output layer**: 10 neurons, corresponding to the 10 MNIST digit classes.

Temporal Encoding and Decoding:

- **Poisson Rate Coding (Input)**: The normalized static pixel intensities of the MNIST images are converted into probabilities. Over a fixed integration window (default $T = 20$ time steps), we generate independent Poisson spike trains for each of the 784 input pixels.
- **Internal Dynamics**: These input spikes propagate through the hidden and output layers, adhering to the biological modifications mentioned previously, using their default parameter values.
- **Spike Count Decoding (Output)**: To retrieve a final classification from the network, we monitor the 10 output neurons over the entire simulation window. The total number of spikes emitted by each output neuron across all T time steps is summed up. The network's final prediction is determined by taking the *argmax* of these cumulative spike counts (i.e., the neuron that fires the most is chosen as the predicted class).

- **Optimisation**: The network is trained using Backpropagation Through Time (BPTT). We use a Fast Sigmoid Surrogate gradient during the backward pass (because the biological step function used to emit a spike (which is non-differentiable) to update. The weights are based on the Cross-Entropy loss of the final spike counts.

We train the model for 50 epochs, and observe the following result for accuracy:

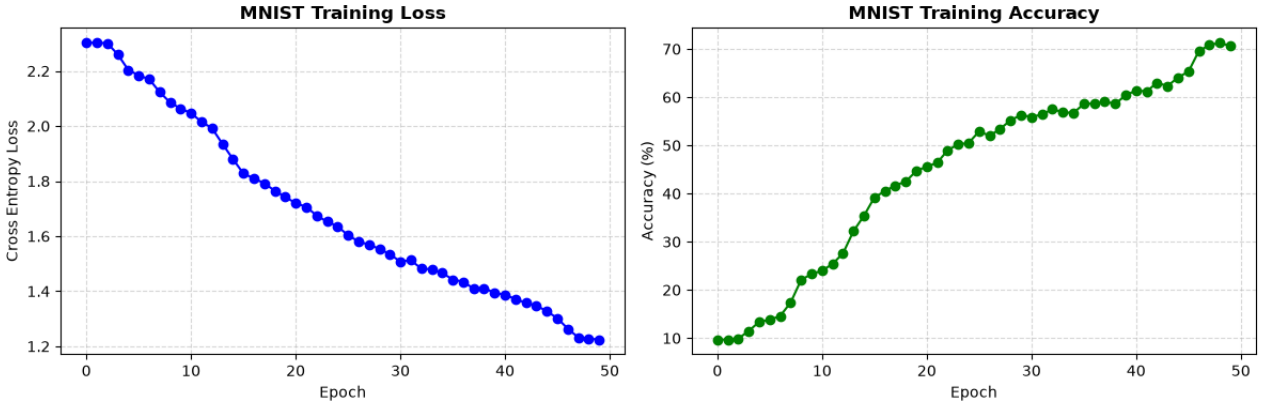
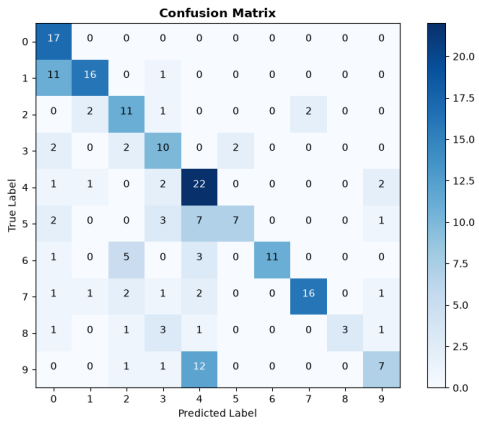


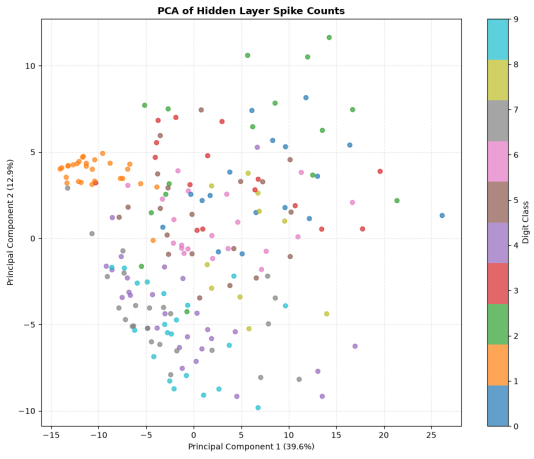
Figure 1.3: Evolution of Loss and Accuracy for Modified Perceptron

Some more interesting plots:

Confusion Matrix and PCA for Hidden Layer



(a) Confusion Matrix



(b) PCA representation for spike counts in the Hidden Layer

Weight Evolution for Hidden Layer

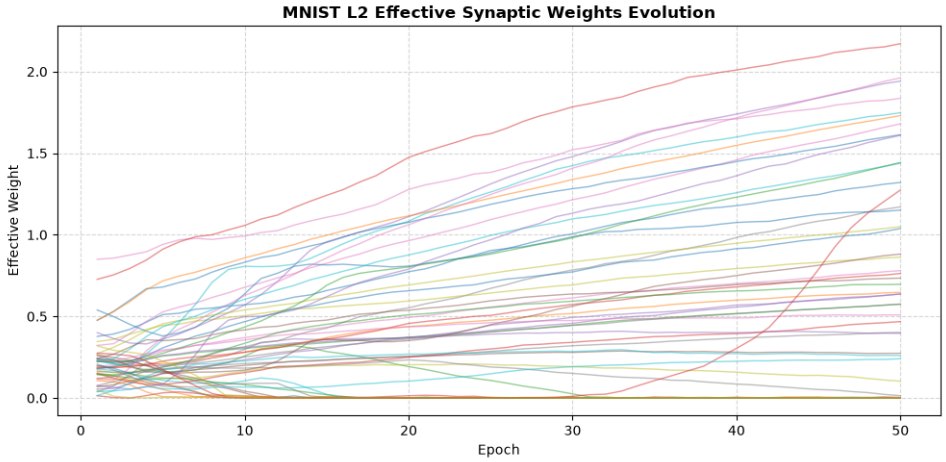


Figure 1.4: Weight Evolution

Notice that during the initial epochs, there was no learning because there were very few spikes.

Overall Weight Evolution Grouped by Output Class

To visualise what the network has learned, we construct the effective receptive field for each of the 10 output neurons. We can approximate the direct mapping from the input pixels to the output classes by computing the matrix product of the two effective weight matrices (for the two layers). This gives a 10×784 matrix, where each row represents the cumulative synaptic pathway from all input pixels to a specific output neuron. By reshaping each of these 10 rows back into a 28×28 spatial grid, we can visually observe the actual shapes and structural patterns that the output neurons have learned to respond to selectively.

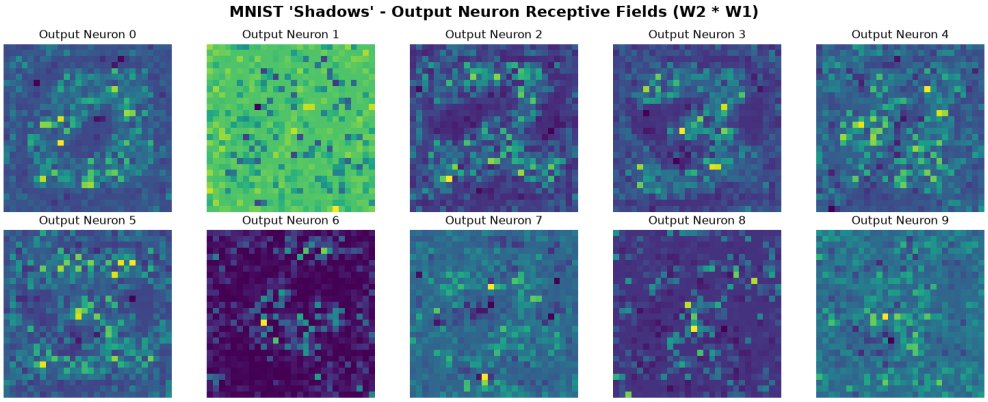


Figure 1.5: Epoch 5

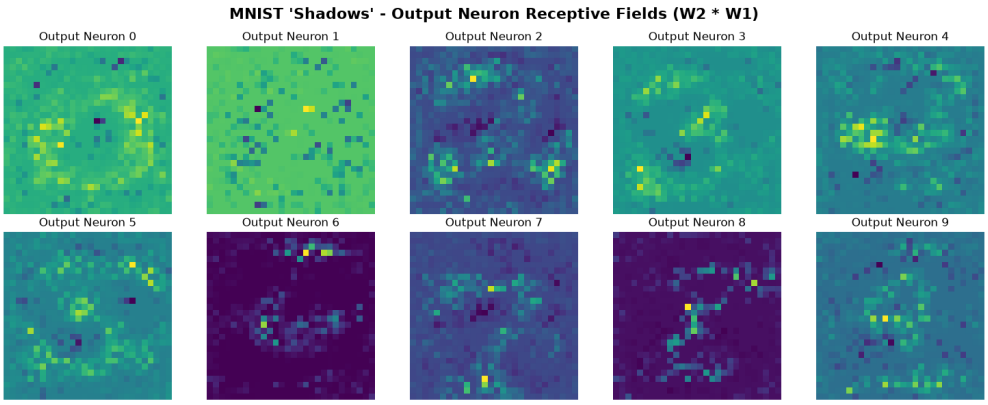


Figure 1.6: Epoch 25

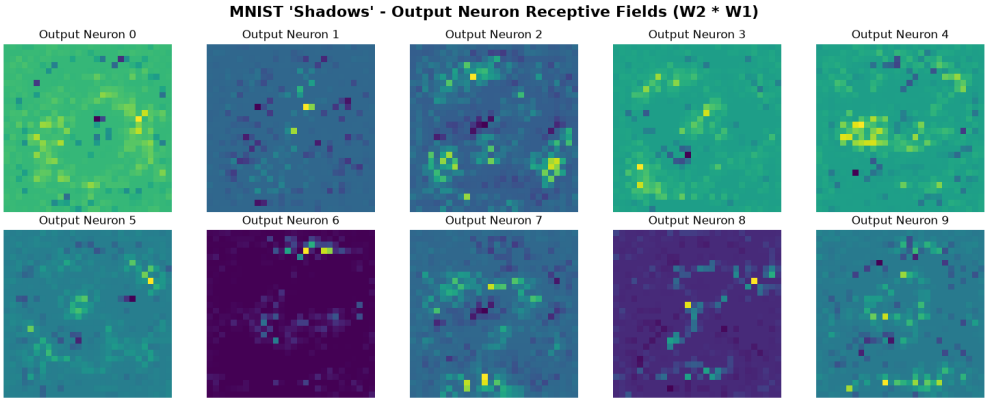


Figure 1.7: Epoch 50

Weight distribution of Excitatory and Inhibitory Synapses

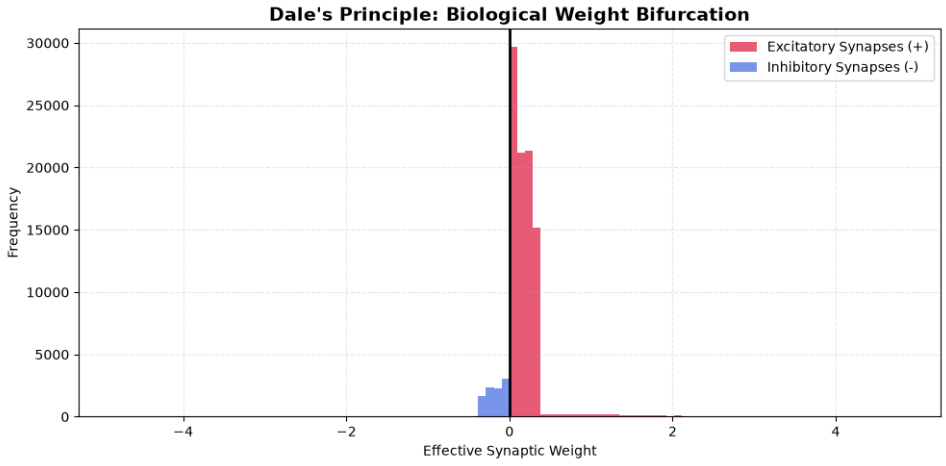


Figure 1.8: $\approx 80\%$ of the synapses are excitatory

1.5.2 Base Perceptron Model

The ANN baseline is a fully connected feed-forward neural network with the same input, hidden, and output dimensions as the Modified Perceptron, allowing for a direct comparison between the two models.

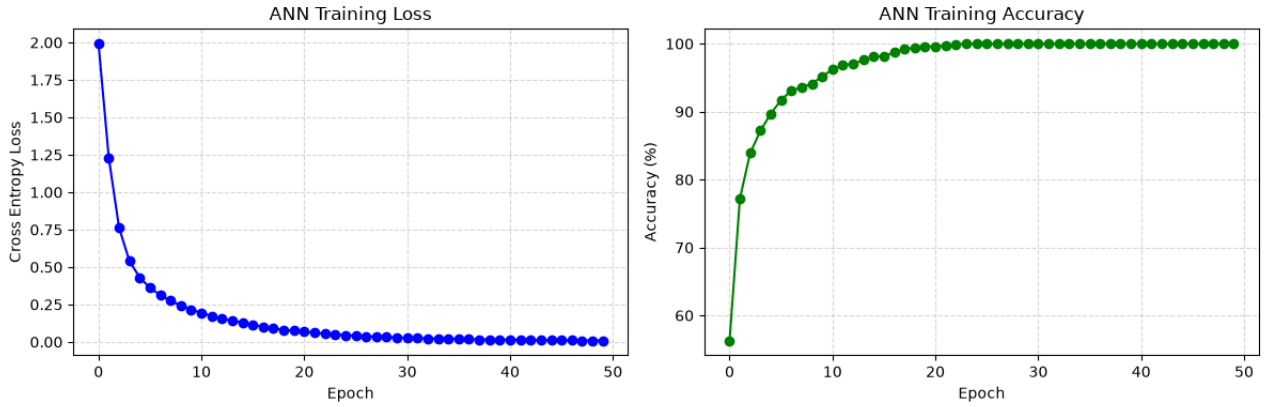


Figure 1.9: Evolution of Loss and Accuracy for Baseline ANN

Weight distribution

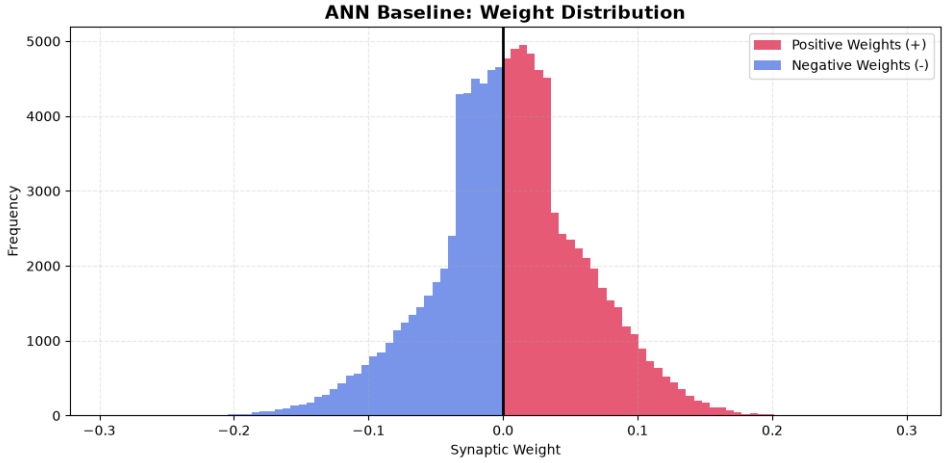


Figure 1.10: Weight distribution is approximately a Gaussian with mean ≈ 0

1.5.3 Weight Distribution: Standard ANN vs. Bio-Inspired Perceptron

In a standard ANN, the synaptic weights naturally form a Gaussian distribution centred around zero. This is because ANN neurons are mathematically unconstrained, as a single neuron can freely learn both positive and negative weights to minimise the loss, causing the overall distribution to cluster tightly around zero.

In our Bio-Inspired Perceptron, this zero-centred Gaussian is destroyed due to separate Excitatory and Inhibitory cells. Due to this, the network's weights are bifurcated into two separate distributions: one strictly positive and one strictly negative. Further, physical distance constraints prune far-away connections; we see a spike of weights around zero, and very few weights at extreme ends.

1.5.4 Effect of Changing the Assumed Parameters

Changing λ

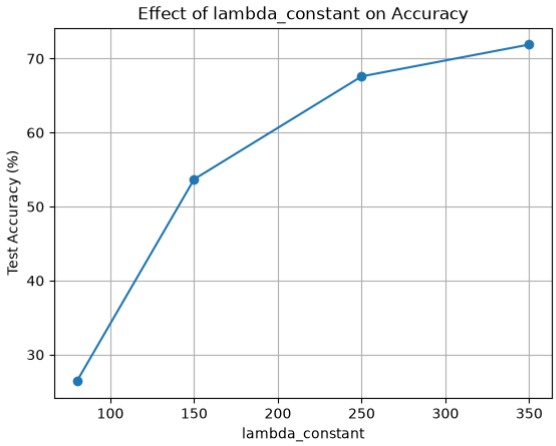


Figure 1.11: Sweeping λ, *ceteris paribus*

This confirms our hypothesis regarding physical structural sparsity. As λ increases from 80.0 μm to 350.0 μm, the accuracy increases monotonically. At low λ values, the exponential attenuation is high, leading to severe signal degradation (and thus vanishing gradients) that structurally isolates the neurons. As λ is increased, the distance-based penalties are reduced, and the architecture behaves increasingly like a dense, unconstrained Artificial Neural Network (ANN).

Changing τ_m

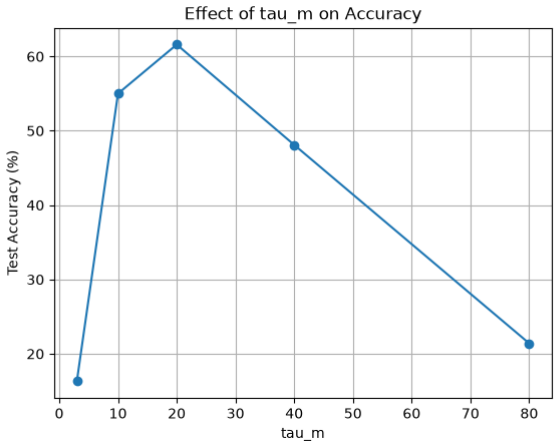


Figure 1.12: Sweeping τ_m, *ceteris paribus*

This sweep reveals an optimal Goldilocks zone. At highly transient timescales (e.g., $\tau_m = 3$ ms), the membrane leaks its accumulated potential too rapidly, preventing the neuron from reaching the firing threshold unless incoming spikes arrive synchronously. At prolonged timescales (e.g., $\tau_m = 80$ ms), the leak mechanism is effectively disabled. The cell acts as a pure, continuous integrator, losing its capacity to discriminate distinct temporal patterns within the spike trains.

Changing U

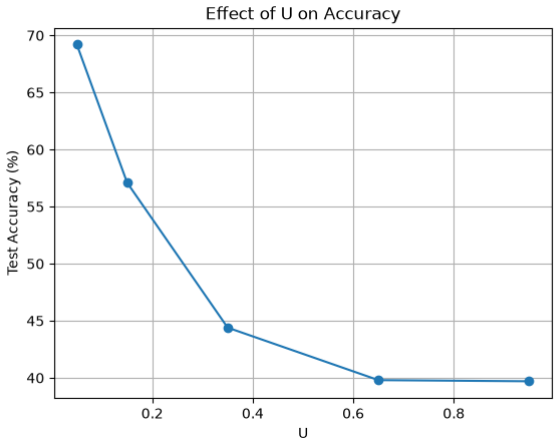


Figure 1.13: Sweeping U, *ceteris paribus*

As predicted, the accuracy decreased as the utilisation factor increased, dropping from 69.2% at U = 0.05 to 39.7% at U = 0.95. A low U disables the Short-Term Depression (STD) mechanism; hence, the synapses experience no fatigue and can fire at high frequencies. However, a high U causes immediate neurotransmitter depletion upon spiking. Although this mechanism guarantees the suppression of runaway excitation, it reduces the communication of spikes through the network, artificially capping its representational capacity and learning rate.

Changing γ

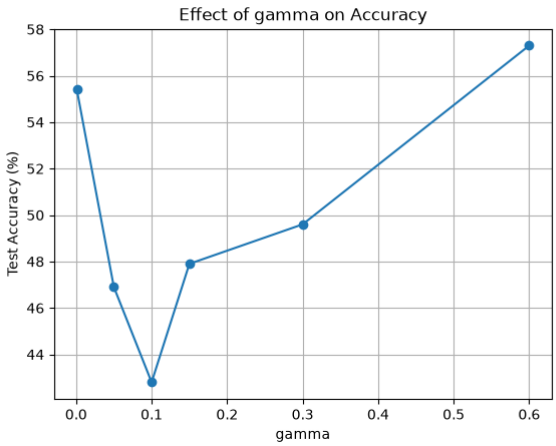


Figure 1.14: Sweeping γ, *ceteris paribus*

The dynamic threshold jump shows non-linear behaviour, forming a U-shaped accuracy curve rather than the predicted failure due to refractory silencing at high γ. The sweep results show that the network found viable alternative optimisation strategies at both extremes. Both a near-zero jump and a massive jump achieve almost equal accuracy. Mid-range values performed the worst. This shows that threshold interference does disrupt learning, but the BPTT optimiser can work with either dense firing regimes (low γ) or sparse, single-spike representations (high γ).

1.5.5 Limitations of the Model

The following are some of the limitations that this model has:

1. **Biological Infeasibility of the Learning Rule:** The backward pass relies on Backpropagation Through Time (BPTT) and surrogate gradients, whereas real brains learn via local rules (such as Spike-Timing-Dependent Plasticity, STDP).

2. **Simplified Neuron Dynamics:** The model abstracts neurons as simple Leaky Integrate-and-Fire (LIF) cells. Actual neurons are far more complex, and there are better models than the LIF model for them (like the Hodgkin–Huxley model).

3. **Assuming Instantaneous Communication:** We use distances to map the attenuations spatially, but we assume instantaneous signal transmission. The length of neurons should introduce some degree of conduction delay. Our Bio-Inspired perceptron does not model that.

4. **Rate Coding vs. Temporal Coding:** The model feeds static MNIST images via Poisson rate coding and decodes via cumulative spike counts over a long time window. True biological networks use temporal coding (e.g., time-to-first-spike), which is significantly faster and more energy-efficient than waiting to accumulate over many timesteps.

1.6 Conclusion

In this assignment, we modified the artificial perceptron to more closely resemble biological neurons by implementing a Bio-Inspired Modified Perceptron. We use Leaky Integrate-and-Fire dynamics, Short-Term Plasticity, Dale's Principle, dendritic attenuation based on physical spatial priors, and dynamic thresholds to show the learning capability on the MNIST dataset.

These biological constraints reduced the model's representational capacity (and hence its classification accuracy) compared to a standard ANN. The trade-off is that mechanisms like neurotransmitter depletion and distance penalties act as natural regularisation and force highly sparse representations.

1.7 Appendix: Plots for Sample Digit 7

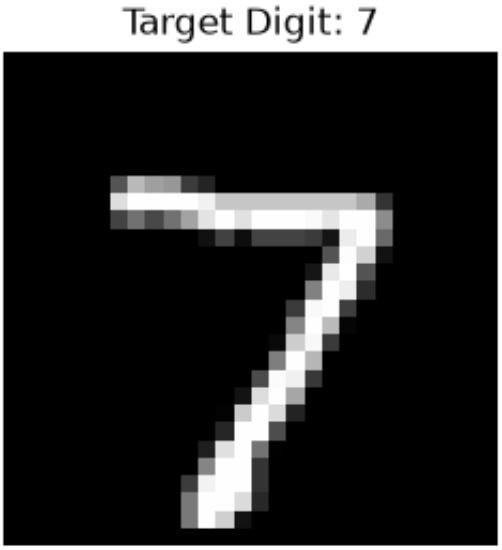


Figure 1.15: Target Image

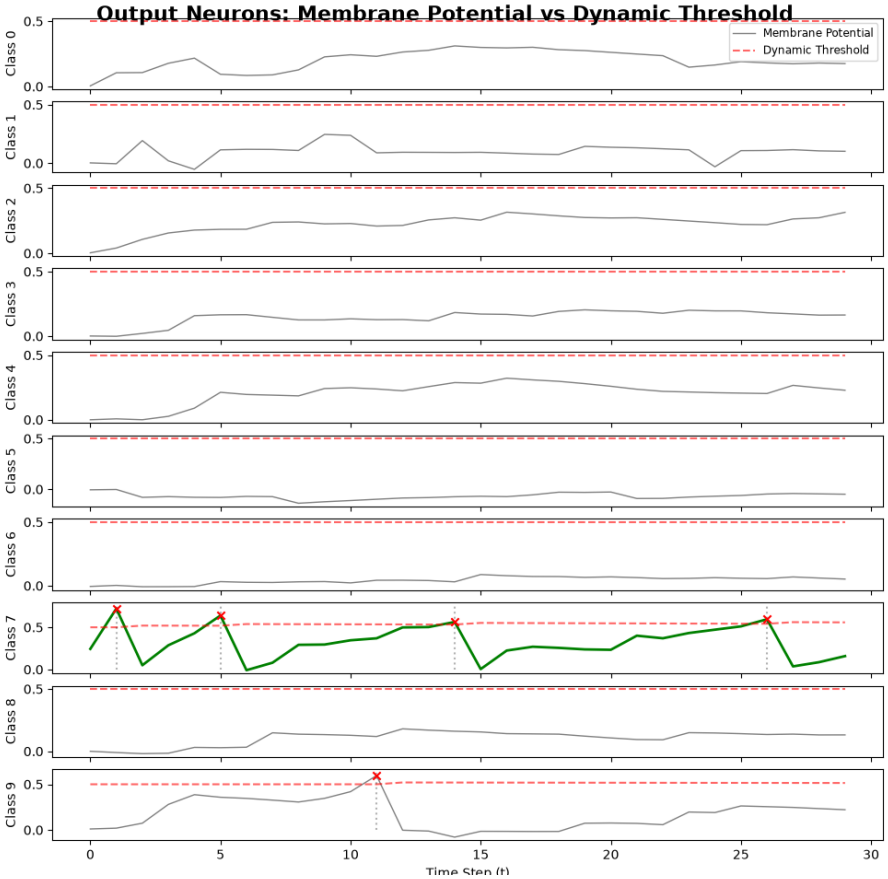


Figure 1.16: Output Neuron Spikes

References

[1] MICrONS Consortium, "Functional connectomics spanning multiple areas of mouse visual cortex," *Nature*, vol. 640, pp. 435–447, 2025.

[2] W. Rall, "Theory of physiological properties of dendrites," *Annals of the New York Academy of Sciences*, vol. 96, no. 1, pp. 1071–1092, 1962.

[3] E. Svensson, J. Apergis-Schoute, G. Burnstock, M. P. Nusbaum, D. Parker, and H. B. Schiöth, "General principles of neuronal co-transmission: Insights from multiple model systems," *Frontiers in Neural Circuits*, vol. Volume 12 - 2018, 2019. [Online]. Available: https://www.frontiersin.org/journals/neural-circuits/articles/10.3389/fncir.2018.00117

[4] M. V. Tsodyks and H. Markram, "The neural code between neocortical pyramidal neurons depends on neurotransmitter release probability," *Proceedings of the National Academy of Sciences*, vol. 94, no. 2, pp. 719–723, 1997.

[5] G. Bellec, D. Salaj, A. Subramoney, R. Legenstein, and W. Maass, "Long short-term memory and learning-to-learn in networks of spiking neurons," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[6] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[7] J. K. Eshraghian, M. Ward, E. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1015–1054, 2023.